than the state as a whole. If the state is large and the component small, this can lead to performance improvements by minimizing component redraws.

For each of our components, then, we need to add the useAppDispatch hook, if needed, and craft a useAppSelector call for the values the component needs. For example, the Subtotal component hook calls look like this:

chapter_12/01/app/javascript/components/subtotal.tsx

```
const Subtotal = (): React.ReactElement => {
  const myTickets = useAppSelector((state) => state.myTickets)
  const dispatch = useAppDispatch()

  const onClear = () => {
    dispatch({ type: "clearHolds" })
  }
  return (
    <>
      <Header>
        <span>Current Tickets Purchased:  </span>
        <span>{myTickets.length}</span>
      </Header>
      <Header>
        <span>Current Tickets Cost:  </span>
        <span>${myTickets.length * 15}.00</span>
      </Header>
      <div className={buttonClass} onClick={onClear}>
        Clear Tickets
      </div>
    </>
  )
}

export default Subtotal
```

Notice the use of useAppSelector to return just the myTickets data and then the references across the component where we had previously referred to data as content.state.myTickets that are changed to just myTickets, because that's what we've assigned the return value of the useSelector hook.

We're not going to look at all the changes to the components here, because the